SECTION 1

The Data API

- **`tf.data.Dataset`:** Represents an ordered stream of elements built from tensors, files, or generators; transform with `map`, `batch`, `shuffle`, `prefetch`, and related methods that each return a new dataset.
- **Scale-out reading:** `list_files` plus `interleave` reads many shards in parallel and mixes lines; `map` with `num_parallel_calls` or `AUTOTUNE` parses and augments on worker threads.
- **Shuffle and repeat:** A large shuffle buffer (up to dataset size) improves IID batches; `repeat` controls epoch boundaries and interacts with `fit()` versus manual `steps_per_epoch`.
- **Performance:** `prefetch` overlaps input preparation with GPU steps; `cache` after parse but before shuffle can eliminate repeated disk reads when RAM allows; `tf.keras` accepts datasets directly in `fit`, `evaluate`, and `predict`.
- **CSV pattern:** `decode_csv` in `map` with training-set means and stds produces normalized numeric batches from text shards without loading full arrays into RAM.
- **Bottleneck signals:** When the GPU waits on batches, `prefetch`, parallel `map`, and faster storage usually beat micro-tuning learning rate.

SECTION 2

The TFRecord Format

- **Records:** Binary length-prefixed records suit large sequential storage; optional GZIP compression helps over networks.
- **Protobuf basics:** Messages serialize with `SerializeToString` / `ParseFromString`; for graphs use TensorFlow's `Example` and `SequenceExample` types with parse ops, not raw `protoc` calls inside `tf.function`.
- **Example schema:** Each instance is a map of named features (`BytesList`, `FloatList`, `Int64List`); `tf.io.parse_single_example` and batch `parse_example` with `FixedLenFeature` / `VarLenFeature` descriptors drive training input.
- **Rich payloads:** JPEG bytes in `BytesList`, serialized tensors, or `SequenceExample` for nested sequences (e.g., sentences as word lists) with ragged follow-up.

SECTION 3

Preprocessing the Input Features

- **Where to preprocess:** Options include offline files, `dataset.map`, Keras preprocessing layers, or TF Transform—trade speed, consistency, and whether statistics are computed once or on the fly.
- **Numeric features:** Standardization via Lambda or `Normalization` layers with `adapt()` on a representative sample; discretization buckets continuous values when nonlinear thresholds help.
- **Categorical features:** Vocabulary lookup plus one-hot for small cardinalities; embeddings (`Embedding` or `embedding_lookup`) scale to large vocabularies and learn task-specific geometry.
- **Text:** `TextVectorization` (or custom tables in `adapt`) maps tokens to indices; bag-of-words and TF-IDF weighting reduce dominance of frequent tokens when sequence order is secondary.
- **Keras preprocessing layers:** `Normalization`, `Discretization`, and `PreprocessingStage` follow the same adapt-then-freeze pattern and should sit at the start of the model graph.

SECTION 4

TF Transform

- **Problem:** Batch preprocessing in Beam/Spark speeds training but risks serving skew if mobile or JS clients reimplement different logic.
- **Idea:** Define `preprocess(inputs)` once with `tensorflow_transform`; analyzers compute global statistics (means, vocabularies); the same graph exports as a TF Function baked into the saved model for identical train and serve behavior.
- **Fit:** Run AnalyzeAndTransformDataset in a Beam pipeline on the full training set, then train on transformed data while keeping the exported transform for production.
- **Install scope:** `tensorflow-transform` ships separately from core TensorFlow and targets TFX production stacks more than notebook-only workflows.

SECTION 5

The TensorFlow Datasets (TFDS) Project

- **`tfds.load`:** Downloads, versions, and documents datasets from MNIST-scale to ImageNet; returns split dictionaries compatible with `tf.data`.
- **Training hygiene:** Apply shuffle, batch, prefetch, and map to `(features, labels)` or use `as_supervised=True` for labeled sets; pair with preprocessing layers or TF Transform when features need adaptation before the core network.
- **When TFRecords help:** Stick with CSV or Parquet until I/O or parsing dominates wall time; serialized `Example` protos then repay the conversion effort.
- **Custom loops:** Chapter 12's manual training patterns consume the same `Dataset` iterators built here batch by batch.